FACE RECOGNITION BASED
ATTENDANCE MANAGEMENT SYSTEM

Detailed Personal Project Report

Python | OpenCV | Haar Cascade | LBPH | CSV



LIVE FACE DETECTION -> RECOGNITION -> ATTENDANCE LOG

Evidence captured from the 11 November 2026 screen recording

Prepared by

P. THARUN SAI RAM

Personal learning and development documentation

Report edition: 11 November 2026

Not a college submission or a formal biometric accuracy certification.

DOCUMENT PROFILE

Purpose, evidence sources and interpretation notes

HOW TO READ THIS REPORT

This report documents a working, learning-oriented attendance prototype developed with Python and OpenCV. It combines a local image dataset, Haar Cascade face detection, LBPH face recognition, a webcam display and CSV attendance logging. The report includes the application screenshots from the new 53-second screen recording supplied on 11 November 2026, two previously supplied test photographs, system diagrams and the source code listing based on the public GitHub repository.

Primary evidence used: the screen recording shows the VS Code project, a successful model-training message, the OpenCV webcam window with the label **Tharun**, and the attendance.csv file with two dated records. The report distinguishes what the recording visibly demonstrates from design explanations and recommendations.

Evidence boundary

The recording shows a successful known-face recognition session. It does not show a separate unknown-person webcam result in this new clip. Unknown-face behaviour is explained from the program logic and the separately supplied held-out sample, not claimed as a newly recorded result.

Code source: the code appendix is a readable, line-numbered presentation of the repository's app.py at the public project URL. Formatting has been normalised for the page layout. The GitHub file remains the authoritative source if it changes later.

Project status: local desktop prototype. Flask is present in requirements.txt, but the demonstrated app is an OpenCV webcam window rather than a deployed Flask dashboard.

Personal use and privacy

This report contains face images supplied for this project. Keep the report and original biometric images in private storage. Do not publish personal face photographs or identifiable attendance records to a public repository without informed permission.

PROJECT CHECKPOINT | QUICK REFERENCE

Evidence

A real screenshot or recorded frame is used when describing a behaviour.

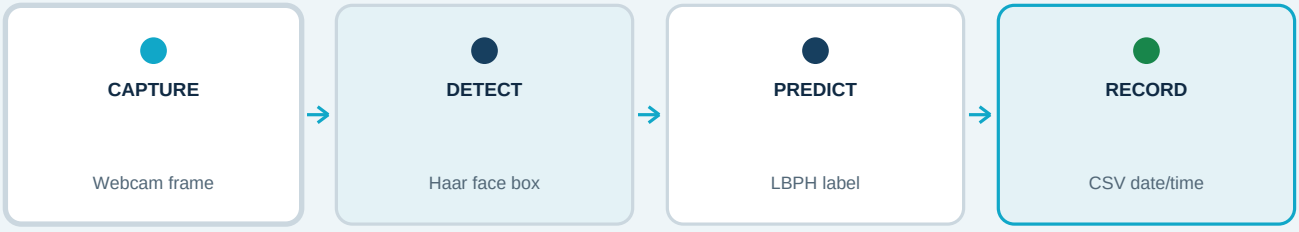
Interpretation

The video proves only the steps that are visible in the captured frames.

Privacy

Personal face data and attendance rows should be kept in restricted storage.

CORE PROCESSING PATH



TAKEAWAY Personal face data and attendance rows should be kept in restricted storage.

EXECUTIVE SUMMARY

What the project does and what the latest demonstration confirms

The Face Recognition Based Attendance Management System is a local computer-vision application designed to recognise an enrolled face from a webcam stream and record a name with the calendar date and time. The core workflow is intentionally compact: collect labelled face images, train an LBPH recogniser, open the webcam, detect a face, classify the face as a known label or Unknown, and append a CSV record when a known person has not already been marked for that date.

DEMONSTRATED CAPABILITIES

- **Model preparation:** the terminal in the recording displays “Face Recognition Model Trained Successfully!” before the camera session.
- **Webcam detection:** a live face is enclosed by a green rectangle in the OpenCV window.
- **Known-face label:** the displayed label over the detected face reads “Tharun”.
- **Session operation:** the console displays “Attendance System Started!” and “Press Q to exit.”
- **CSV records:** the attendance.csv screenshot shows the Name, Date and Time columns and records for 10 November 2026 and 11 November 2026.

CORE TOOLS

Python
Application logic and CSV handling

OpenCV
Camera capture and face processing

Haar Cascade
Frontal-face detection

LBPH
Appearance-based face classification

CSV
Local attendance history

GitHub
Version-controlled source repository

Overall result

The latest recording provides clear evidence of the recognised-face webcam path and local attendance-file output. It is a functional demonstration, not a statistically measured recognition benchmark.

PROJECT CHECKPOINT | QUICK REFERENCE

Input
Person folders provide the face samples used for training.

Processing
Haar Cascade detects and LBPH predicts a class label.

Output
A name and timestamp are appended to the CSV for a first daily mark.

ABSTRACT AND KEYWORDS

Project summary

Manual attendance processes require an operator to identify participants and write or type their presence into a register or spreadsheet. A computer-vision workflow can automate part of this routine when a registered person's face is visible to a camera and can be matched reliably against prior samples. This project implements a learning prototype using Python and OpenCV. Haar Cascade detects face regions in grayscale frames, while the Local Binary Patterns Histograms (LBPH) recogniser compares the detected face crop with labelled training examples stored in person-specific folders.

When the recogniser returns a label with a distance below the configured threshold, the program resolves the label to a person's folder name. It checks the existing attendance.csv entries for a record with the same name and current date. If the person has not yet been marked for that day, a row containing the name, date and time is appended. A non-accepted prediction is displayed as “Unknown”.

The new demonstration recording shows the project open in Visual Studio Code, the model training message, a live webcam window with a green face box and the label “Tharun”, and the CSV file containing dated attendance entries. The current implementation is a local desktop program. It does not include liveness detection, a production web dashboard, encryption or centrally managed user access.

KEYWORDS

- Face Recognition
- Face Detection
- OpenCV
- Python
- Haar Cascade
- LBPH
- Webcam
- Attendance Management
- CSV Logging
- Computer Vision

Interpretation

An LBPH prediction is a similarity-based classification within the enrolled dataset. The label “Unknown” means the prediction did not meet the configured acceptance rule; it does not independently establish a person's real-world identity.

PROJECT CHECKPOINT | QUICK REFERENCE

Detection

Locate a face rectangle in a grayscale image.

Recognition

Compare the normalised crop with trained LBPH samples.

Attendance

Use the accepted folder label to create a dated record.

CORE PROCESSING PATH

CAPTURE → Webcam frame

DETECT → Haar face box

PREDICT → LBPH label

RECORD → CSV date/time

TAKEAWAY Use the accepted folder label to create a dated record.

TABLE OF CONTENTS

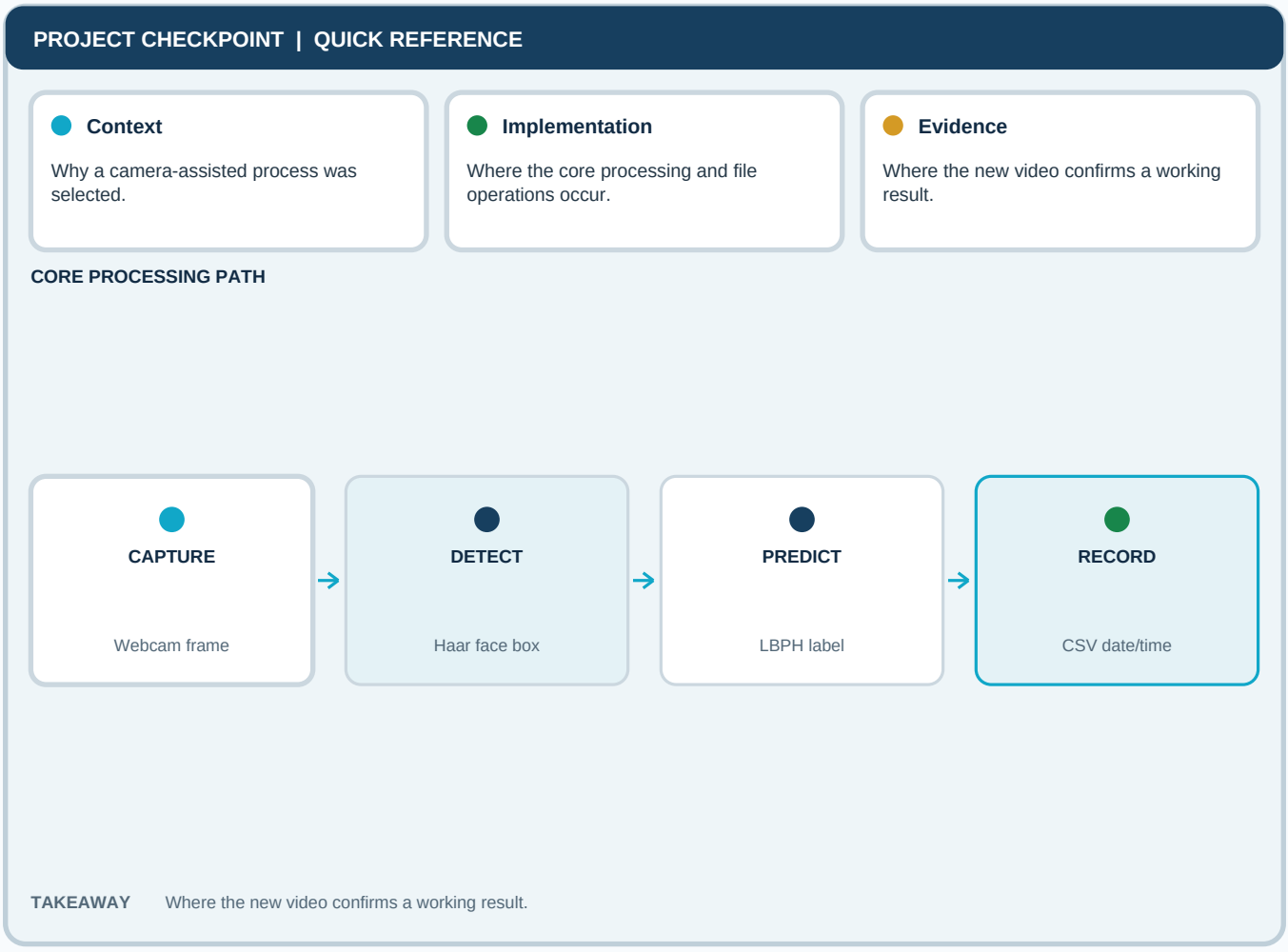
Report navigation

Pages	Sections
1-5	Report profile, executive summary, abstract and contents
6-10	Introduction, problem definition, objectives, alternatives and technology stack
11-15	Requirements, project structure, architecture and face-image samples
16-20	Preprocessing, Haar Cascade, LBPH, training and decision threshold
21-25	New video evidence, console startup, CSV records, attendance logic and code walkthrough
26-30	Runtime steps, testing, result analysis, privacy and limitations
31-34	Troubleshooting, setup, GitHub project and code appendix introduction
35-38	Complete source-code listing, future scope, conclusion and references

The page references above identify the major sections. Code pages are grouped at the end so the explanatory chapters can be read independently. The screenshots are placed beside the relevant technical discussion rather than collected only in a final image appendix.

Reading path

For a quick project explanation, read pages 3, 12, 21, 22, 23 and 37. For implementation details, read pages 16-20 and the source-code appendix.



1. INTRODUCTION

Context and project motivation

Attendance recording is a frequent administrative task in educational institutions, workplaces, training centres and small teams. A manual register is easy to start but places the full burden of identity checking and record entry on a human operator. When many people arrive at once, the process may become slow, and manual writing can introduce missed or repeated entries.

Face recognition offers one possible automation path. A camera captures a frame, the software locates the face region, and a trained model estimates which enrolled label most closely resembles the detected face. Attendance software can then apply a rule before storing a record. The goal of this project is not to claim that facial recognition is perfect; it is to demonstrate the end-to-end integration of a camera, a trained model and a simple attendance file.

PROJECT MOTIVATION

- Convert classroom learning in Python and computer vision into a runnable application.
- Understand the difference between face detection and face recognition.
- Learn how images can be arranged into labelled training folders.
- Connect live webcam processing to date-and-time attendance recording.
- Practice environment setup, dependency management and source-code versioning.

The selected architecture is suitable for a small local demonstration because it relies on familiar Python libraries and a simple CSV data file. The trade-off is that the system remains dependent on the quality and variety of the local face images and the camera conditions.

Project type

A personal learning and development project. It is not represented as a production attendance product or a formal academic submission.

PROJECT CHECKPOINT | QUICK REFERENCE

Motivation

Build an end-to-end project instead of isolated code exercises.

Integration

Connect camera input, model prediction and file handling.

Learning

Practice Python, OpenCV, testing and version control.

CORE PROCESSING PATH



TAKEAWAY Practice Python, OpenCV, testing and version control.

2. PROBLEM DEFINITION

Need, challenges and project response

Problem: a manual attendance process requires an operator to identify each person, enter the person's details and maintain a record. This approach can consume time and can produce typing mistakes, missed entries or inconsistent timestamps. The project explores a camera-assisted alternative for a small, enrolled set of people.

SPECIFIC CHALLENGES

- **Repeated manual entry:** the operator must type or write each attendance event.
- **Identity checking:** the process relies on the operator to recognise the participant.
- **Record consistency:** the date and time must be entered in a consistent format.
- **Duplicate records:** the same person may be entered more than once in a day if no checking rule exists.
- **Auditability:** paper entries can be difficult to filter or export.

PROJECT RESPONSE

The prototype captures webcam frames and uses OpenCV to detect a face. The LBPH recogniser predicts a label and distance. If the prediction passes the threshold, the application checks a set of names already marked for the current date. A new CSV row is written only if the name is not already in that set. The application therefore demonstrates automatic record entry and a simple same-day duplicate rule.

Design boundary

A face-based match is not proof of a person's presence in a secure sense: the current program has no liveness detection and could be vulnerable to a photograph or displayed image. Supervision and consent remain necessary.

PROJECT CHECKPOINT | QUICK REFERENCE

Manual risk

Typing may introduce omissions or inconsistent entries.

Camera path

Automate a narrow recognition and record-entry step.

Oversight

Keep human review for errors and exceptions.

CORE PROCESSING PATH



TAKEAWAY Keep human review for errors and exceptions.

3. AIM, OBJECTIVES AND SCOPE

Project definition

Aim: to develop a local webcam-based attendance application that detects a face, recognises an enrolled label, and records the person's name with the date and time in a CSV file.

OBJECTIVES

- Load labelled samples from person-specific folders under known_faces.
- Detect faces in the training images and resize face crops to a common size.
- Train an LBPH face recogniser using the prepared face samples and numeric labels.
- Read frames from the default webcam and detect visible faces in real time.
- Display a green rectangle and a person's folder name when a prediction is accepted.
- Display the label Unknown when the prediction is not accepted.
- Append one attendance event per person per current date using a CSV file.

INCLUDED AND EXCLUDED SCOPE

Included in the prototype	Not currently included
Local webcam input	Cloud camera or multi-site deployment
Frontal face detection	Robust profile / full-pose recognition guarantee
LBPH-based label prediction	Deep neural embedding pipeline
CSV name/date/time log	Central database with accounts and permissions
Same-day name set check	Tamper-proof audit or cryptographic log
Console + OpenCV window	Complete administrator web dashboard

Success criteria

A successful demonstration means the software starts, the model trains, the webcam displays a detected face with the expected label in the recorded conditions, and attendance rows are visible in the CSV file.

PROJECT CHECKPOINT | QUICK REFERENCE

Aim

Recognise an enrolled face and log a first attendance event.

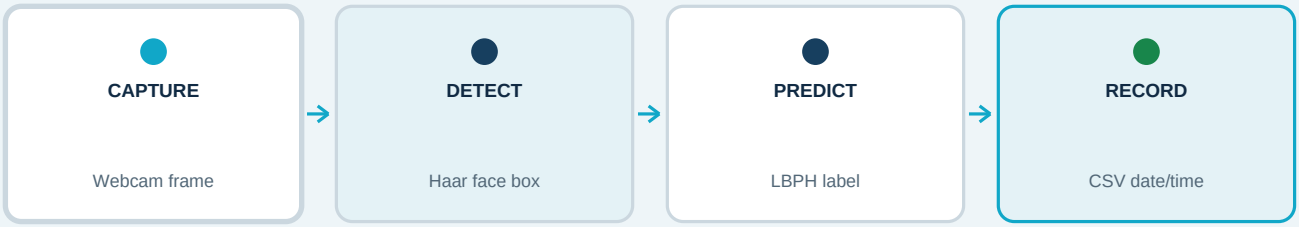
Included

Webcam, Haar Cascade, LBPH and CSV operations.

Excluded

No central dashboard, liveness or production access controls.

CORE PROCESSING PATH



TAKEAWAY No central dashboard, liveness or production access controls.

4. EXISTING AND PROPOSED APPROACH

Comparison of manual and camera-assisted attendance

Aspect	Manual attendance	Current prototype
Identity check	Human visual recognition	LBPH label prediction
Input	Register or spreadsheet	Webcam frames
Entry operation	Written or typed	CSV row appended by code
Timestamp	Entered or noted manually	System date and time
Duplicate handling	Depends on operator	Name is checked against today's set
Unknown person	Operator decides	Displayed as Unknown if prediction fails threshold
Security	Physical control of register	Local files; no access-control layer

The proposed system automates only the narrow part of the workflow that the prototype implements. Human oversight remains important for exceptions, poor lighting, changed appearance and cases where the face is not visible. The CSV file is convenient for a demonstration but is not a substitute for a managed database when multiple operators or devices are involved.

EXPECTED OPERATIONAL BENEFITS

- Less repetitive manual typing for the enrolled person.
- Consistent date and time format generated by the computer.
- A simple visible indication when the face label is accepted.
- An attendance history that can be opened in a spreadsheet editor.

Fair comparison

The system does not eliminate human oversight. It changes the record-entry mechanism from manual typing to a camera-based prediction that should still be checked.

PROJECT CHECKPOINT | QUICK REFERENCE

Manual

Operator identifies and records each participant.

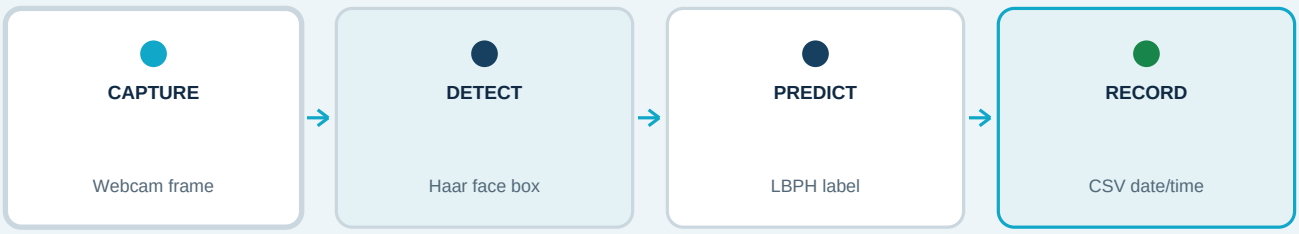
Proposed

The program predicts an enrolled label from a webcam frame.

Safeguard

Keep a manual correction path for misclassifications.

CORE PROCESSING PATH



TAKEAWAY Keep a manual correction path for misclassifications.

5. TECHNOLOGY STACK

Libraries and their responsibilities

Component	Role in this project	Notes
Python 3.12	Runs the application	Selected local environment shown in the recording
OpenCV (opencv-contrib-python)	Camera, image operations and LBPH	The contrib package exposes cv2.face
NumPy	Label array for model training	Used with recognizer.train
Haar Cascade XML	Frontal-face detection	OpenCV packaged cascade is referenced in app.py
CSV module	Read and append attendance rows	Stores Name, Date and Time
datetime	Current date and time	Formats the attendance timestamp
Visual Studio Code	Editor and terminal	Shown in the supplied recording
Git and GitHub	Source control and public repository	Project source is published online
Pandas / openpyxl / Pillow / Flask	Additional installed requirements	Not all are directly used by the current app.py

The requirements file in the repository lists opencv-contrib-python, numpy, Flask, pandas, openpyxl and Pillow. The currently demonstrated app.py uses OpenCV, NumPy, Python's csv module, os and datetime. Flask is a dependency in the environment list, but the recorded application is not a Flask web application.

Why the contrib package matters

The LBPH recogniser is accessed through cv2.face. Installing the OpenCV contrib distribution is necessary for this API in the selected implementation.

PROJECT CHECKPOINT | QUICK REFERENCE

Runtime

Python executes the application and control flow.

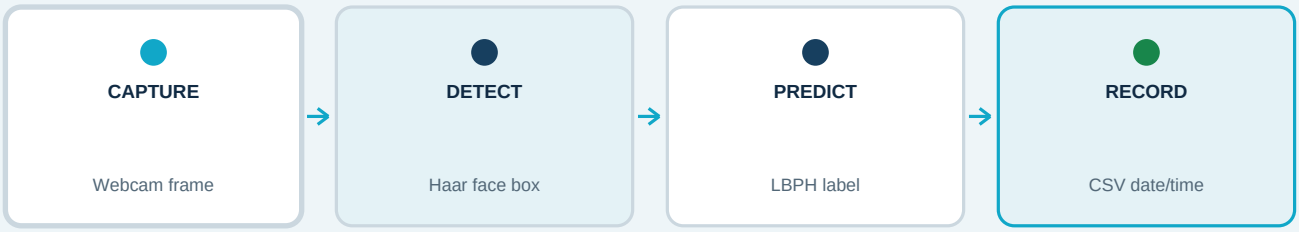
Vision

OpenCV and the contrib face module provide detection and LBPH.

Storage

CSV and datetime provide a lightweight local record.

CORE PROCESSING PATH



TAKEAWAY CSV and datetime provide a lightweight local record.

6. HARDWARE AND SOFTWARE REQUIREMENTS

Local development environment

HARDWARE

- Windows laptop or desktop with a working webcam.
- A stable surface or laptop stand to keep the camera still.
- Even front lighting so facial details remain visible.
- Keyboard to start and stop the program; the Q key closes the camera window.

SOFTWARE

Software item	Project use
Windows 10 / 11	Desktop operating system
Python 3.12	Runtime environment used in the successful local setup
Visual Studio Code	Editing app.py and running the terminal
Git for Windows	Local version-control commands
OpenCV contrib build	Haar Cascade and LBPH face recognition
Project virtual environment (.venv)	Isolated project dependencies

The latest recording's VS Code status bar indicates Python 3.12.10 in the project virtual environment. This matters because the application packages were installed in that environment; choosing a different interpreter can lead to ModuleNotFoundError even when OpenCV is installed elsewhere.

Environment rule

Activate the project's .venv and run python app.py from the project folder. Avoid launching the script with a global Python 3.13 or 3.14 interpreter unless the same dependencies have been installed there.

PROJECT CHECKPOINT | QUICK REFERENCE

Camera

Use a working webcam and stable laptop placement.

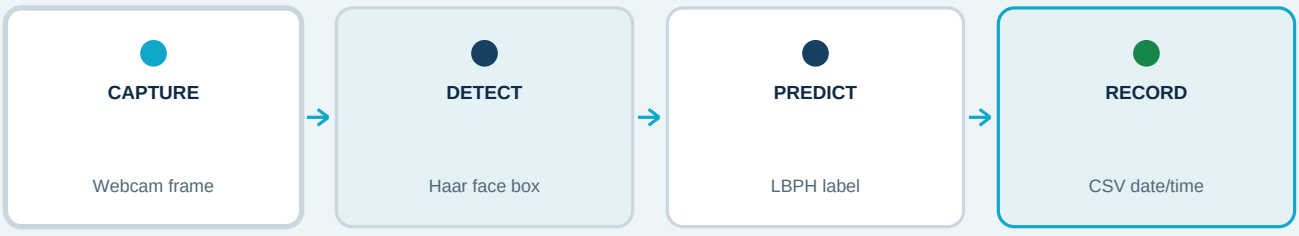
Interpreter

Run the project in its Python 3.12 virtual environment.

Lighting

Even front light helps preserve facial details.

CORE PROCESSING PATH



TAKEAWAY Even front light helps preserve facial details.

7. PROJECT DIRECTORY STRUCTURE

Files visible in the latest recording

PYTHON SOURCE

```
Face-Recognition-Attendance-System/
|-- .venv/                          Local Python environment
|-- known_faces/                    Registered face images
|   |-- Tharun/                     Person-specific training folder
|-- .gitignore                      Ignore rules for local-only data
|-- app.py                          Main application source
|-- attendance.csv                  Attendance records
|-- haarcascade_frontalface_default.xml
|                                   Local cascade file visible in Explorer
|-- README.md                       Repository overview
|-- requirements.txt                Python package list
```

The application reads folder names under known_faces as person labels. For example, the folder name Tharun becomes the display name when the face is classified with that folder's numeric label. To enrol another person, a new folder with that person's chosen label would be created and filled with suitable face images.

- Keep the training folder structure consistent: one identity per folder.
- Keep .venv local; it should not be uploaded to GitHub.
- Keep personal face photographs private unless the people pictured have agreed to publication.
- Keep attendance.csv local or replace it with a synthetic sample before making the repository public.

Cascade path note

The repository app.py uses cv2.data.haarcascades to reference OpenCV's bundled frontal-face XML. A separate XML file is visible in the project folder screenshot; it may be an additional copy and is not the path used by that line of code.

PROJECT CHECKPOINT | QUICK REFERENCE

Dataset

known_faces contains one folder per enrolled label.

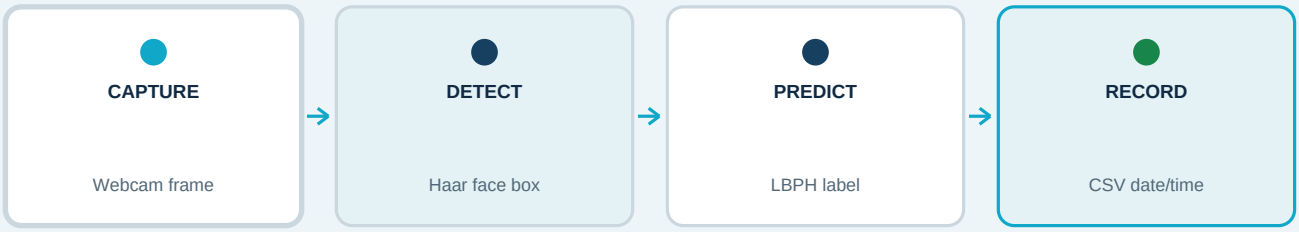
Program

app.py contains the training and camera loop.

Local data

Keep .venv and personal attendance records out of public source control.

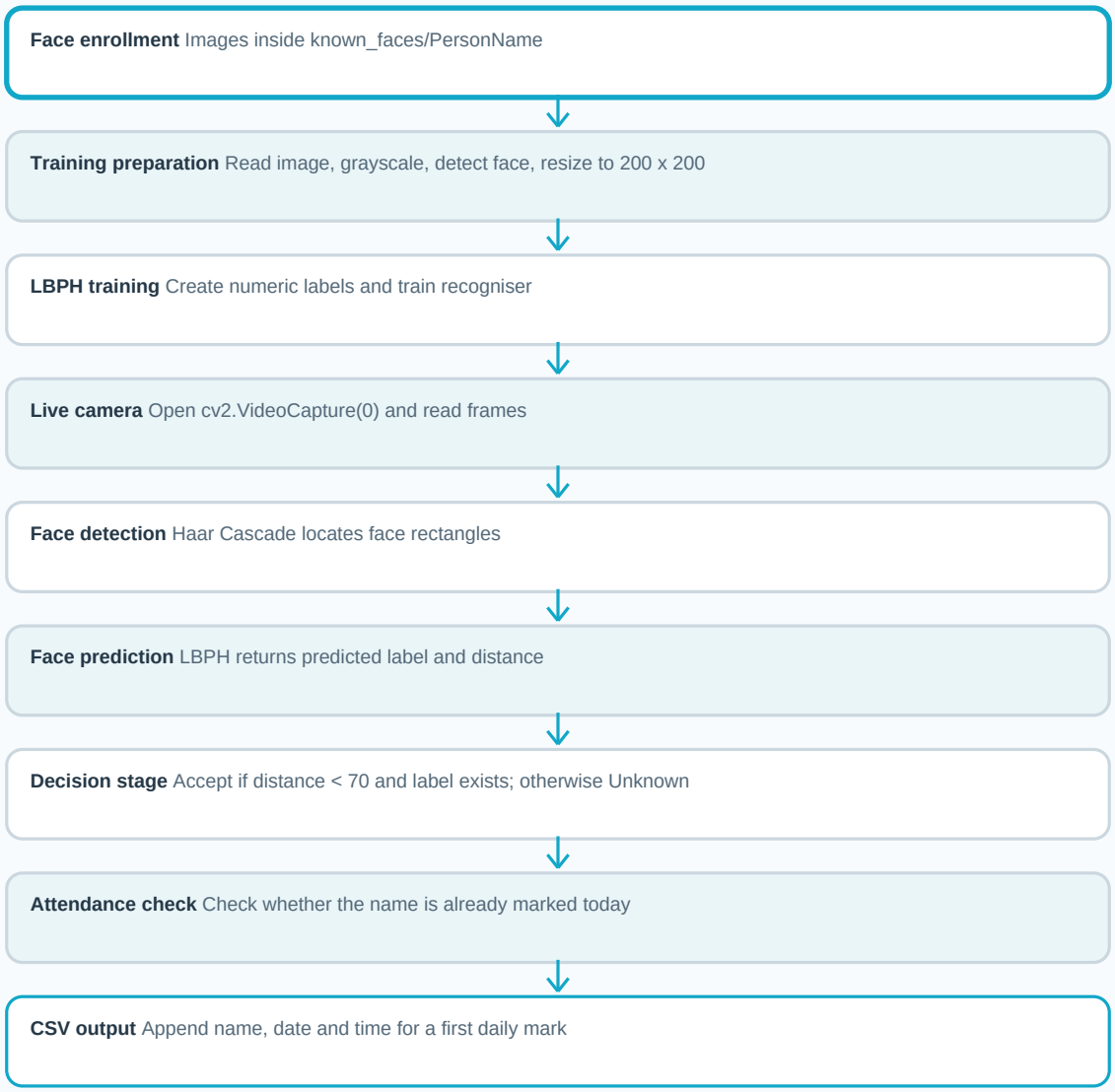
CORE PROCESSING PATH



TAKEAWAY Keep .venv and personal attendance records out of public source control.

8. SYSTEM ARCHITECTURE

Main components and data flow



Data flow summary

Training data is read from the local folder tree. Live webcam frames are temporary processing inputs. Only a recognised name and timestamp are written to attendance.csv by the current implementation.

PROJECT CHECKPOINT | QUICK REFERENCE

Training path	Live path	Record path
Prepare face crops from labelled sample folders.	Capture frames and predict labels in a repeated loop.	Check the daily set before writing attendance.csv.

9. FACE DATASET: REGISTERED SAMPLE

The first photograph supplied for the project

The first photograph supplied in the conversation is designated as the registered or known-face sample for this report. The actual training dataset is stored under the person folder in known_faces. The repository code uses the folder name as the identity label and uses the image files inside that folder to prepare the LBPH training samples.



Figure 9.1 - supplied registered-face sample designated for the Tharun folder

How enrollment works: save suitable photographs inside known_faces/Tharun/. During training, the program reads each image, converts it to grayscale, runs the face detector and extracts the largest face rectangle. The face crop is resized to 200 by 200 pixels before it is added to the training list.

- Use clear, front-facing samples with eyes and facial outline visible.
- Include modest variation in expression and camera distance while keeping the subject clear.
- Avoid motion blur, backlighting, heavy occlusion and very small face crops.
- Store training images locally and restrict access to the folder.

Privacy note

Face images are personal biometric-related data. Do not publish the real training folder in a public GitHub repository unless there is a clear, informed reason and permission from the person pictured.

PROJECT CHECKPOINT | QUICK REFERENCE

Enrollment	Preparation	Label
Store only clear, consented photographs in the person folder.	Detect, crop and resize each usable face to 200 x 200.	The folder name becomes the person name in the display.

10. HELD-OUT / UNREGISTERED TEST IMAGE

Second photograph supplied for the report

The second photograph was designated by the user as an unknown-face test image. For a valid unregistered-input test, this image must be kept outside every folder used to train the recogniser. The software can then be run with the photograph or a live pose resembling it as a test input, and the output can be checked against the acceptance threshold.



Figure 10.1 - supplied side-facing image designated as a held-out input

Important interpretation: the two supplied photographs appear to show the same person in different poses. The word “unknown” in this test means that the image is not enrolled in the training folder. It does not mean the image belongs to a different person. Depending on pose, lighting and LBPH distance, the recogniser could still match it to the enrolled label.

Correct test setup

Keep the held-out photo out of known_faces/Tharun/. Run a controlled test and record the actual prediction. A correct unknown test is based on the model output, not only on the fact that an image is missing from the training folder.

PROJECT CHECKPOINT | QUICK REFERENCE

Held-out input

Keep the test photograph outside the training folders.

Prediction

Check whether the model accepts or rejects its similarity score.

Meaning

Unknown means not accepted as an enrolled class, not a verified identity.

CORE PROCESSING PATH

CAPTURE → DETECT → PREDICT → RECORD

CAPTURE	DETECT	PREDICT	RECORD
Webcam frame	Haar face box	LBPH label	CSV date/time

TAKEAWAY Unknown means not accepted as an enrolled class, not a verified identity.

11. IMAGE ACQUISITION AND PREPROCESSING

From colour pixels to a standard face crop

The system receives images in colour through OpenCV. It converts an image to grayscale before applying the Haar Cascade detector. Grayscale reduces each pixel to intensity information and is suitable for the selected classical detection and recognition operations. The detector returns one or more face rectangles, each described by x, y, width and height.

PREPROCESSING SEQUENCE

Read image or frame cv2.imread for dataset photos; cap.read for webcam

↓

Convert to grayscale cv2.cvtColor(..., cv2.COLOR_BGR2GRAY)

↓

Detect candidate faces detectMultiScale returns face rectangles

↓

Select largest face max area rectangle in each training image

↓

Crop face region gray[y:y+h, x:x+w]

↓

Resize cv2.resize(face, (200, 200))

↓

Send to LBPH Store the normalized face crop and its label

The same 200 by 200 size is used during training and prediction. Matching input dimensions help keep the representation consistent, but resizing cannot correct all pose, blur or lighting differences.

Practical capture guidance

Keep the camera close to eye level, face a soft light source and hold a neutral, comfortable expression. Repeatedly moving the camera or changing the lighting during a test can affect the recognition distance.

PROJECT CHECKPOINT | QUICK REFERENCE

Grayscale	Crop	Resize
Convert the image to a single intensity channel.	Extract the detected face rectangle from the frame.	Use the same 200 x 200 dimensions for training and prediction.

12. FACE DETECTION WITH HAAR CASCADE

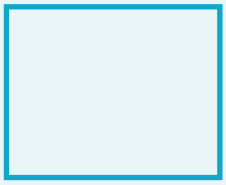
Locating a face in the image

Haar Cascade is a classical object-detection approach. It uses a cascade of simple image features and a sequence of classifier stages to identify regions that resemble a frontal face. In this project, the OpenCV frontal-face cascade is loaded from the package data directory. The detector is applied to grayscale images and returns rectangular regions that may contain faces.

KEY PARAMETERS USED IN THE REPOSITORY

Parameter	Current value	Purpose
scaleFactor	1.3	Image pyramid reduction between detection scales
minNeighbors	5	Neighbour requirement to filter weak candidate rectangles
minSize	Not explicitly set	OpenCV default minimum size applies
Cascade XML	haarcascade_frontalface_default.xml	Trained frontal-face classifier

A detected face is not the same as a recognised identity. Detection answers “where is a possible face region?” It does not answer “who is this person?”. That second question is delegated to the LBPH recogniser after the detected rectangle has been cropped and resized.



Face rectangle

The detector returns x, y, width and height.

The program crops the grayscale face area.

Limitations

The selected cascade is most reliable in conditions similar to its training examples, especially a reasonably frontal face. Strong side angles, low light, occlusion or blur may reduce detection quality.

PROJECT CHECKPOINT | QUICK REFERENCE

Detector
Haar Cascade returns candidate face rectangles.

Parameters
The current code uses scaleFactor 1.3 and minNeighbors 5.

Boundary
A frontal-face detector may be less reliable for side poses.

13. FACE RECOGNITION WITH LBPH

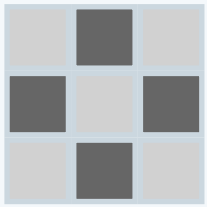
Local Binary Patterns Histograms

LBPH is a classical appearance-based face-recognition technique. It describes local texture patterns by comparing each pixel with neighbouring pixels, forming binary patterns that are accumulated into histograms. The trained recogniser stores a representation for the labelled face samples. For a new face crop, the model predicts the nearest label and returns a distance-like confidence value.

HOW THE PREDICTION IS USED

- The detected face is resized to 200 by 200 pixels.
- recognizer.predict(face) returns a numeric label and a confidence/distance value.
- The application checks whether the confidence value is below 70.
- The numeric label is mapped to a folder name through the names dictionary.
- If either the threshold test or label lookup fails, the displayed name is Unknown.

Conceptual local texture comparison



Neighbouring intensity comparisons become binary patterns.

Local patterns are summarised into histograms.

The model compares the test crop with stored samples.

Threshold caution

The value 70 is a project parameter, not a universal accuracy standard. Its behaviour should be measured using separate known and unknown test images under multiple lighting and pose conditions.

PROJECT CHECKPOINT | QUICK REFERENCE

Local pattern

LBP compares neighbouring pixel intensities.

Histogram

Local patterns are summarised for comparison.

Prediction

LBPH returns a label and distance-like score.

CORE PROCESSING PATH

CAPTURE — Webcam frame → DETECT — Haar face box → PREDICT — LBPH label → RECORD — CSV date/time

TAKEAWAY LBPH returns a label and distance-like score.

14. MODEL TRAINING WORKFLOW

Preparing labels and face samples

At application startup, the recogniser is created using cv2.face.LBPHFaceRecognizer_create(). The program iterates through the subfolders in known_faces. Each folder name is stored in a dictionary against an integer label. Each readable image is converted to grayscale and scanned by the face detector. The largest detected face is cropped and resized. The face crop is appended to the training list and the matching numeric label is appended to the label list.

Training object	Meaning
faces	List of normalized grayscale face crops
labels	Integer class label for each face crop
names	Dictionary mapping integer labels to folder names
label_id	Next numeric label assigned to a person folder

TRAINING OPERATIONS

- Create the LBPH recogniser.
- Read each person folder and assign its numeric class label.
- Read images and skip files OpenCV cannot decode.
- Detect faces and skip images where no face is detected.
- Crop and resize the largest detected face.
- Call recognizer.train(faces, np.array(labels, dtype=np.int32)).

The terminal message “Face Recognition Model Trained Successfully!” in the screen recording is direct evidence that execution reached the post-training line. It does not by itself measure the model's accuracy on unseen images.

Data quality rule

Each person's folder should contain multiple clear images. More images do not automatically mean better results; use varied but usable photographs and keep a separate set for testing.

PROJECT CHECKPOINT | QUICK REFERENCE

Read

Load images from each identity folder.

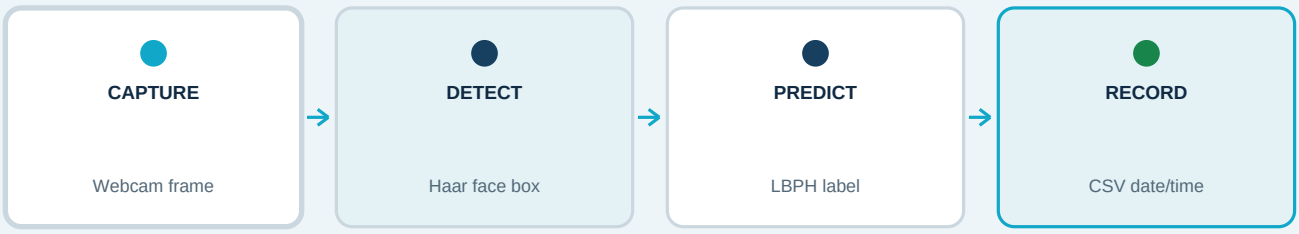
Train

Pair face crops with integer labels and train LBPH.

Verify

The success message means the training call completed.

CORE PROCESSING PATH



TAKEAWAY The success message means the training call completed.

15. ACCEPT / UNKNOWN DECISION

Recognition threshold and label mapping

LBPH returns a predicted label and a distance value. The current repository code accepts the prediction when confidence is less than 70 and the predicted label exists in the names dictionary. If accepted, the folder name is used as the display label and the rectangle is green. Otherwise, the label becomes Unknown and the rectangle is red.

Condition	Display	Action
confidence < 70 and label in names	Folder name, green box	Check daily attendance set
Any other prediction	Unknown, red box	Do not write attendance row

WHY THE THRESHOLD MATTERS

A threshold creates a trade-off. If it is too permissive, the model may accept a face that should be unknown. If it is too strict, it may reject the correct enrolled person. The suitable value depends on the camera, image quality, dataset variety and expected conditions. Threshold tuning should be performed on held-out samples and should record both false-acceptance and false-rejection outcomes.

- Known test: use an enrolled image not used for training and check whether the correct name appears.
- Unknown test: use a test image kept outside every enrolled folder and check whether Unknown appears.
- Repeat each test at different distances and lighting levels.
- Record the predicted label and distance for each test; do not rely only on visual impression.

Important

A pose change of the same person can be classified as Unknown if the image differs sufficiently from the stored samples. Conversely, an unregistered image may be incorrectly assigned to a known label if its distance passes the threshold.

PROJECT CHECKPOINT | QUICK REFERENCE

Threshold

The project accepts a prediction when distance is below 70.

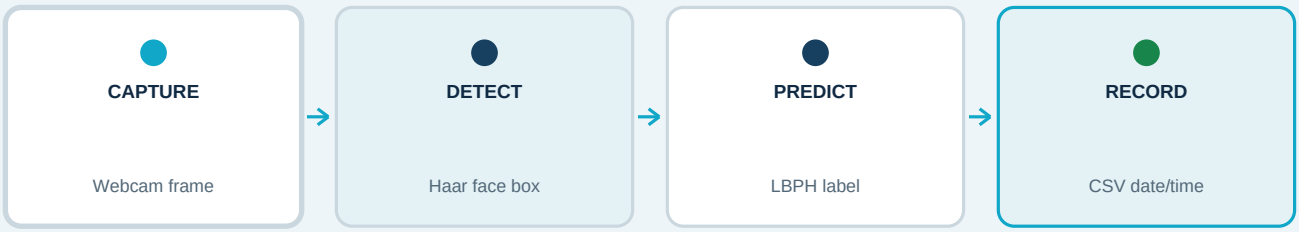
Trade-off

A loose threshold risks false accepts; a strict one risks rejects.

Validation

Calibrate with held-out known and unknown examples.

CORE PROCESSING PATH



TAKEAWAY Calibrate with held-out known and unknown examples.

16. NEW SCREEN RECORDING: LIVE RECOGNITION

Webcam output captured on 11 November 2026

The supplied 53-second screen recording shows the OpenCV window titled “Face Recognition Attendance System”. A face is enclosed by a green rectangle, and the label above it reads “Tharun”. The result is visible in the live webcam feed while Visual Studio Code and the project terminal remain open behind the camera window.

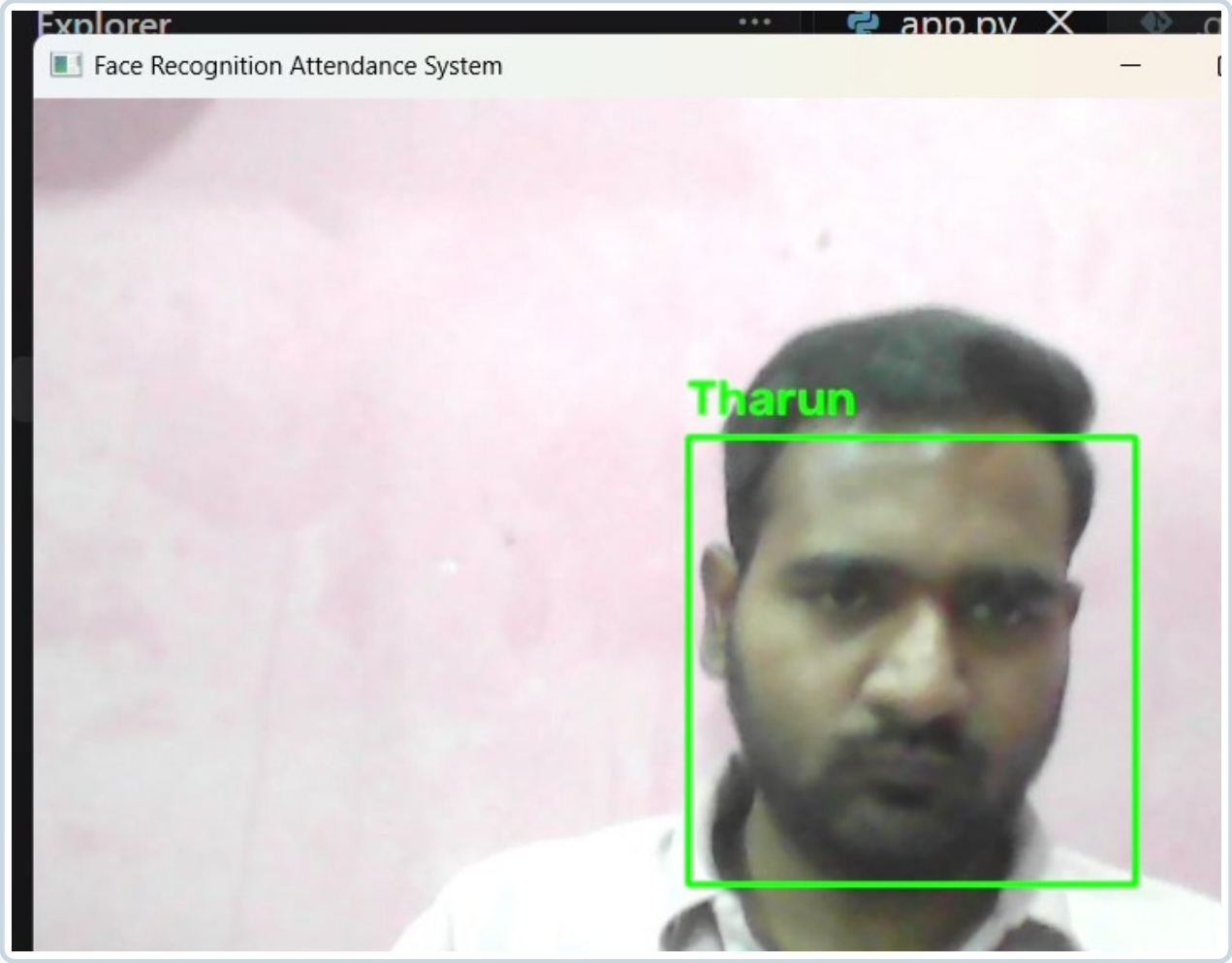


Figure 16.1 - enlarged webcam-window crop from the new screen recording

The green rectangle and name label are consistent with the accepted-known branch of the source code. The screenshot demonstrates that the camera opens, a face is detected and the recogniser returns the enrolled name for this captured condition.

Observed outcome

Known-face recognition for the label Tharun is visible in the new recording. The screenshot is taken from the user-supplied video; it is not a generated mock-up.

PROJECT CHECKPOINT | QUICK REFERENCE

Observed	Recognised	Scope
Green rectangle surrounds a detected live face.	The visible webcam label reads Tharun.	This clip demonstrates a known-face case, not a full accuracy study.

17. NEW SCREEN RECORDING: STARTUP MESSAGES

Model training and webcam start

The video shows the VS Code integrated terminal after the command python app.py is run from the activated virtual environment. The visible output includes the messages “Face Recognition Model Trained Successfully!”, “Attendance System Started!” and “Press Q to exit.” These lines mark the transition from the model setup stage to live frame processing.

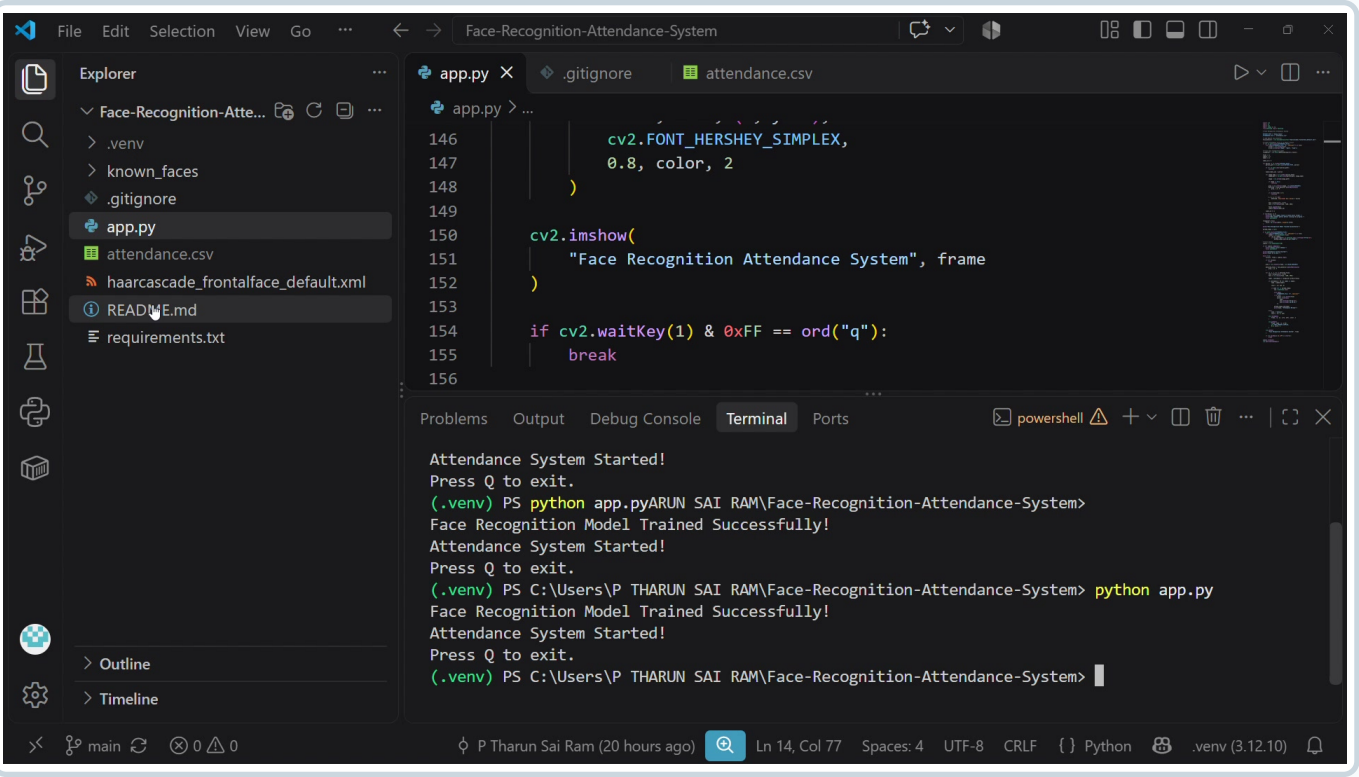


Figure 17.1 - project source and terminal output from the screen recording

WHAT THE MESSAGES ESTABLISH

- **Training reached completion:** the code passed through recognizer.train without terminating at that point.
- **Webcam stage started:** the application proceeded to the message printed immediately before the camera loop.
- **Quit instruction:** Q is the configured keyboard key to exit the OpenCV window.
- **Environment:** the video shows the project terminal operating with the local .venv active.

Evidence limitation

A successful startup message confirms the application entered the camera stage. It does not prove recognition accuracy across all users or conditions.

PROJECT CHECKPOINT | QUICK REFERENCE

Terminal	Model	Control
The project command is run from the repository folder.	Training success is printed before camera capture.	Press Q in the OpenCV window to exit.

18. NEW SCREEN RECORDING: ATTENDANCE CSV

Visible name, date and time records

The attendance.csv screenshot shows the header Name,Date,Time and two rows for Tharun. One entry is dated 10 November 2026 at 12:49:11; the second is dated 11 November 2026 at 10:26:49. This gives a concrete example of the CSV format and confirms that the local file contains attendance records across two dates.

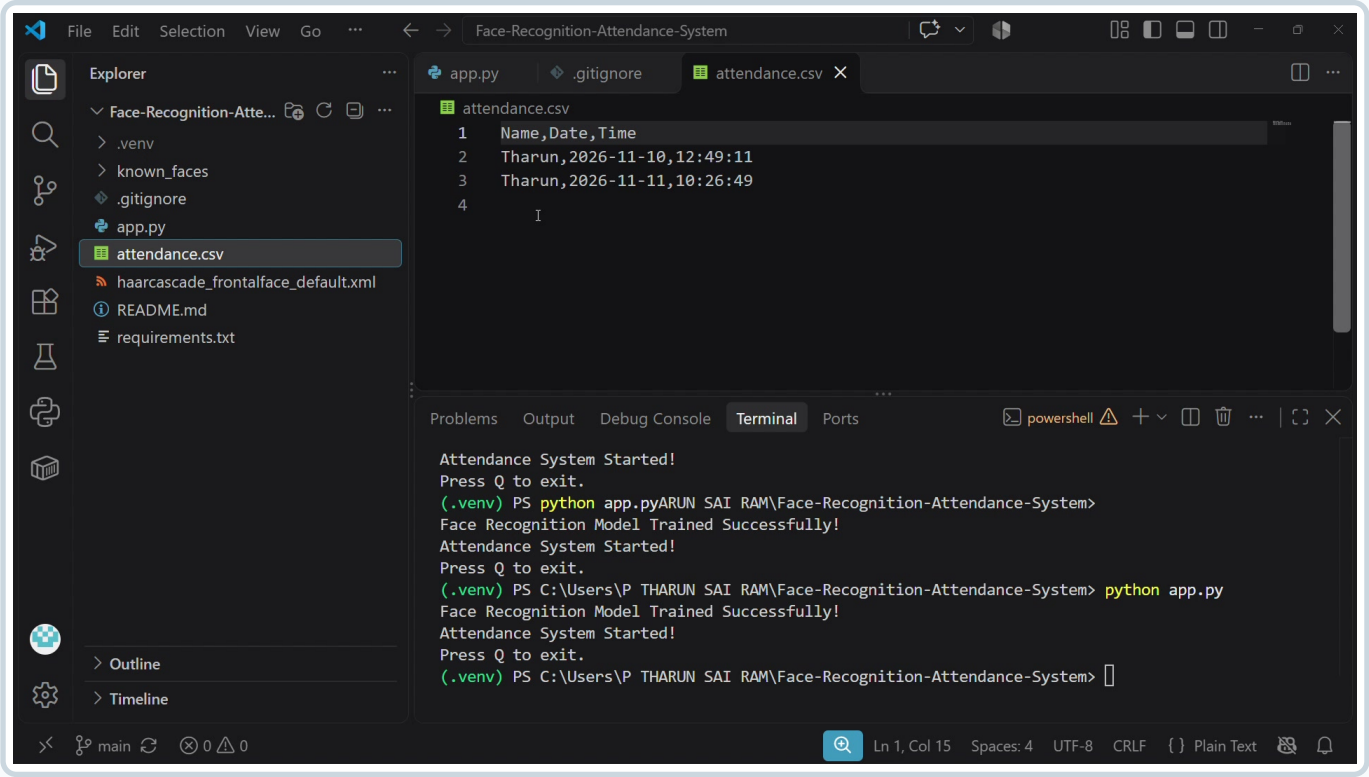


Figure 18.1 - attendance.csv open in VS Code during the new demonstration

VISIBLE SAMPLE VALUES

Name	Date	Time
Tharun	2026-11-10	12:49:11
Tharun	2026-11-11	10:26:49

Interpretation

The file screenshot demonstrates saved records. The two records are on separate dates, not duplicate records for the same date. The program's daily set is intended to block another new entry for a name already present for the current date.

PROJECT CHECKPOINT | QUICK REFERENCE

Schema
The CSV header is Name, Date and Time.

Records
Two dated entries for Tharun are visible in the screenshot.

Persistence
Rows remain in the local file after the program closes.

19. ATTENDANCE FILE DESIGN

CSV schema and record lifecycle

The application uses a comma-separated values file named attendance.csv. If the file does not exist when the program starts, the application creates it and writes a header row with three columns: Name, Date and Time. For a first accepted recognition on the current day, the program appends the folder name, current date and current time as a new row.

Column	Example	Meaning
Name	Tharun	Recognised folder label
Date	2026-11-11	Current local date in YYYY-MM-DD format
Time	10:26:49	Local system time in HH:MM:SS format

ADVANTAGES OF CSV FOR A PROTOTYPE

- Human-readable in a text editor or spreadsheet program.
- Easy to create and append without a database server.
- Portable for a small single-machine demonstration.
- Simple to inspect when verifying a successful attendance run.

OPERATIONAL CAUTIONS

CSV is not a transactional multi-user database. Concurrent applications could write at the same time, and a user with file access can edit or delete rows. For a classroom or workplace deployment, the storage layer should be redesigned with access controls, backups and an audit trail.

Record correctness

The file should be reviewed periodically. If old test rows are present, preserve them as historical evidence or create a clean test copy before demonstrating the first-mark and duplicate-mark cases.

PROJECT CHECKPOINT | QUICK REFERENCE

Name

The person folder name is stored in the first column.

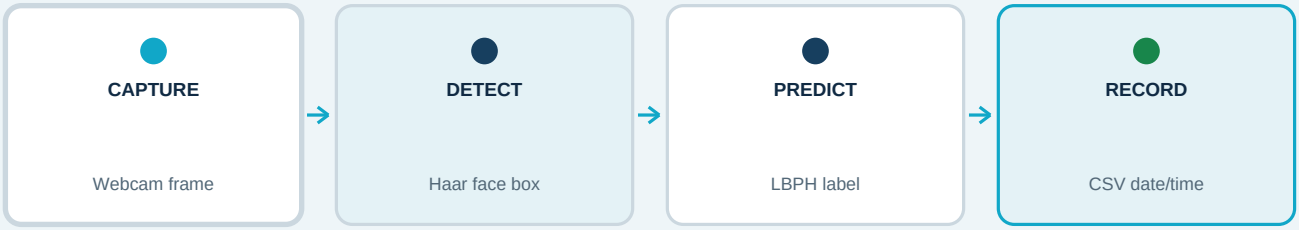
Date

The application formats the local calendar date.

Time

The timestamp uses hours, minutes and seconds.

CORE PROCESSING PATH



TAKEAWAY The timestamp uses hours, minutes and seconds.

20. SAME-DAY DUPLICATE PREVENTION

How the application remembers a daily mark

The repository code creates an in-memory set named marked_today. At startup it reads attendance.csv using csv.DictReader and adds each row's Name to the set when the row's Date equals the current date. When a face is recognised, the program checks whether the name is already in the set. If it is not present, a row is appended and the name is added to the set immediately.

PYTHON SOURCE

```
001  marked_today = set()
002  for row in attendance.csv:
003      if row['Date'] == today:
004          marked_today.add(row['Name'])
005
006  if recognised_name not in marked_today:
007      append(name, today, current_time)
008      marked_today.add(recognised_name)
009  else:
010      do not append another row
```

WHY THE SET IS UPDATED IMMEDIATELY

After a new row is written, marked_today.add(name) runs before the next webcam frame is processed. The program therefore does not need to reopen the CSV file for every camera frame. This keeps the daily check simple and fast for a small local list.

Scope of this rule

The set is held in memory for the current application run and is rebuilt from the CSV at startup. It is a simple single-process protection, not a database-level uniqueness constraint. A multi-device version needs stronger storage safeguards.

PROJECT CHECKPOINT | QUICK REFERENCE

Load state

Read existing rows and collect names for today's date.

Check

Only a name absent from the set is eligible for a new row.

Update

Add the name to the set immediately after writing.

CORE PROCESSING PATH



TAKEAWAY Add the name to the set immediately after writing.

21. RUNTIME ALGORITHM

From webcam frame to attendance row

Start application Load libraries, file paths and cascade

↓

Train recogniser Read enrolled samples and train LBPH

↓

Load daily state Read today's names from attendance.csv

↓

Open camera cv2.VideoCapture(0)

↓

Capture frame Read image from camera

↓

Detect faces Grayscale + Haar Cascade

↓

Predict face label LBPH predicts integer label and distance

↓

Decision Accept known label or show Unknown

↓

Attendance check If today's set does not contain the name, append row

↓

Display and repeat Draw box, show frame, stop when Q is pressed

Resource cleanup

After the loop ends, camera.release() frees the webcam and cv2.destroyAllWindows() closes OpenCV windows. The finally-style cleanup is not used in the compact repository source, so a future refactor could make cleanup more robust on unexpected exceptions.

PROJECT CHECKPOINT | QUICK REFERENCE

Start	Loop	Stop
Load the detector and train the recogniser.	Read frames, detect faces and predict the label.	Exit on Q and release camera resources.

22. TESTING STRATEGY AND OBSERVED RESULTS

Functional checks from development and recording

Test	Expected behaviour	Evidence/status
Application launch	App starts without import failure	New video shows startup output
Model training	LBPH training completes	Training-success line visible
Webcam access	Camera window opens	Live window visible
Face detection	Face rectangle drawn	Green box visible
Known face	Name label appears	“Tharun” visible in video
CSV record structure	Name, Date, Time header	Header visible in CSV
Timestamp records	Rows include date and time	Two dated rows visible
Unknown face	Unknown label if prediction fails threshold	Logic documented; not shown in new clip
Same-day repeat	No additional row for same name/date	Implemented with marked_today set; requires controlled retest

Test results are based on the submitted recording and the project code visible in the public repository. The recording verifies the known-face path and the CSV file. It does not provide a measured accuracy score, false-acceptance rate, false-rejection rate or an unknown-person demonstration in this clip.

Recommended next test

To document unknown recognition conclusively, keep the second supplied test image outside known_faces, run the program, capture the displayed result and save a screenshot. Repeat a known-face scan twice on the same date and confirm the CSV does not gain a second row.

PROJECT CHECKPOINT | QUICK REFERENCE

Verified

Training, webcam display, Tharun label and CSV are visible.

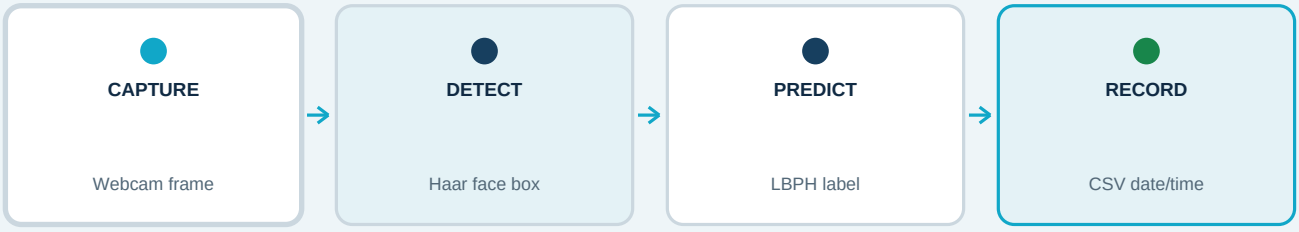
Not shown

An unknown-person result is not visible in the new clip.

Next test

Record known, unknown and repeat-scan cases separately.

CORE PROCESSING PATH



TAKEAWAY Record known, unknown and repeat-scan cases separately.

23. RESULT ANALYSIS

What the latest demonstration establishes

The latest recording provides practical evidence that the integrated application can be started from the activated project environment and that the live face window can display the enrolled label. The green box and Tharun label indicate that a face region was detected and the recogniser accepted the prediction for the recorded frame. The CSV screenshot shows that the application maintains a small dated attendance history.

QUALITATIVE OUTCOME

- **Integration:** the camera, detection, recognition and record-storage components are connected in one application.
- **Usability:** the webcam output presents an easily understood rectangle and name label.
- **Traceability:** the CSV file can be opened and inspected in VS Code.
- **Repeat sessions:** the visible records show entries on two dates, demonstrating that the file persists between runs.

WHAT WAS NOT MEASURED

No formal test dataset, identity-balanced evaluation, precision/recall calculation, latency measurement or lighting study is included in the supplied recording. The project therefore should not claim a percentage accuracy. To produce such a figure, define the test set, ground-truth labels, threshold, and a repeatable measurement procedure first.

Fair reporting

Use wording such as “successfully demonstrated known-face recognition in the recorded test environment.” Avoid claims such as “100% accurate” unless supported by a defined and repeatable evaluation.

PROJECT CHECKPOINT | QUICK REFERENCE

Functional

The integrated path starts and presents a recognition result.

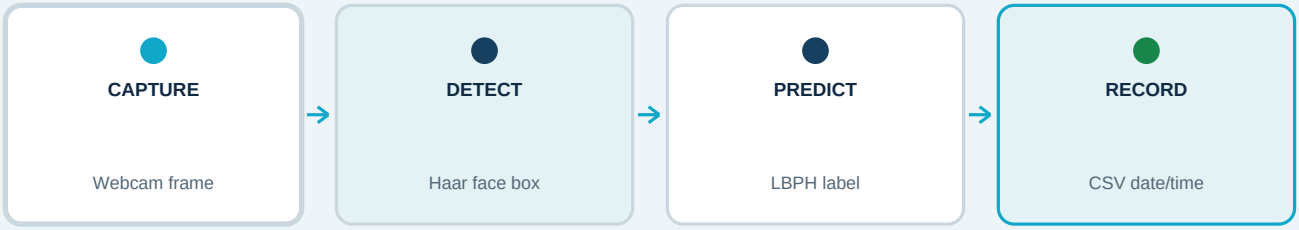
Traceable

The CSV file provides inspectable attendance rows.

Unmeasured

No formal accuracy percentage is claimed.

CORE PROCESSING PATH



TAKEAWAY No formal accuracy percentage is claimed.

24. PERFORMANCE AND QUALITY FACTORS

Conditions that affect recognition

The application processes frames from the laptop webcam and applies detection and recognition to the faces returned by the cascade. Its practical speed depends on camera resolution, the number of visible faces, CPU performance, and the number of training examples. LBPH is computationally lighter than many modern deep-learning face-embedding pipelines, which makes it approachable for a small local prototype.

Factor	Possible effect	Practical response
Lighting	Shadows or backlight change facial texture	Use even front light
Pose	Side profile can alter local patterns	Collect pose-diverse samples; test separately
Distance	Small faces produce fewer details	Keep face reasonably close to camera
Blur	Motion blur reduces feature detail	Hold still and keep camera stable
Occlusion	Masks, hands or hair hide regions	Capture clear unobstructed examples
Dataset imbalance	One person has many more samples	Use consistent sample quality and count

A useful test log should include the lighting condition, pose, distance, model distance returned by LBPH, expected identity and actual output. This allows the threshold to be assessed from observed evidence rather than chosen by guesswork.

Measurement recommendation

Collect a small test matrix with multiple known and unknown samples and record outcomes. Report the counts of correct known matches, correct unknown rejections, false accepts and false rejects.

PROJECT CHECKPOINT | QUICK REFERENCE

Environment

Lighting and camera placement change the input quality.

Dataset

Pose and sample variety affect the LBPH comparison.

Evaluation

Measure false accepts and false rejects on held-out samples.

CORE PROCESSING PATH



TAKEAWAY Measure false accepts and false rejects on held-out samples.

25. PRIVACY AND RESPONSIBLE USE

Handling facial images and attendance information

The project uses face photographs to train and test a recognition model and stores names and timestamps in an attendance file. Face photographs can reveal sensitive biometric-related information. Attendance records can reveal a person's routine or presence at a location. These files should be treated as personal data and protected accordingly.

RECOMMENDED SAFEGUARDS

- Obtain clear permission before enrolling a person's face or using their photo for a test.
- Explain the purpose of capture, where images are stored and who can access them.
- Keep real known_faces photographs on the local machine or in a restricted private repository.
- Do not upload personal attendance.csv records to a public GitHub repository.
- Use synthetic example records in public documentation.
- Delete training and test images when they are no longer required.
- Restrict access to the laptop account and back up only what is necessary.

PUBLIC REPOSITORY NOTE

The project repository is public. Before future commits, review the contents of known_faces and attendance.csv. If real face photographs or real attendance rows are tracked by Git, remove them from the public history where appropriate and add ignore rules so they are not reintroduced. A simple .gitignore protects future commits but does not erase data already committed.

Do not use this prototype as a sole decision-maker

A recognition error can incorrectly record or reject attendance. Provide a manual correction path and retain human oversight.

PROJECT CHECKPOINT | QUICK REFERENCE

Consent

Explain the purpose before collecting face photographs.

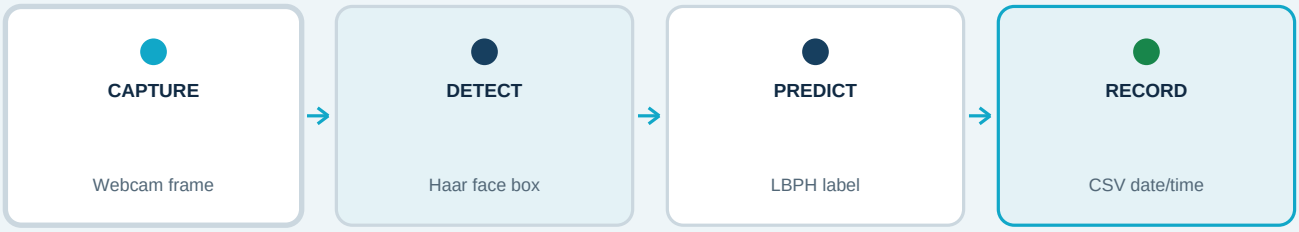
Access

Limit who can view training images and attendance rows.

Retention

Delete sample data when it is no longer required.

CORE PROCESSING PATH



TAKEAWAY Delete sample data when it is no longer required.

26. LIMITATIONS AND RISK ANALYSIS

Current prototype boundaries

Limitation	Why it matters	Possible improvement
No liveness detection	A printed or displayed photo may be accepted	Add tested liveness / anti-spoofing
Frontal-face cascade	Side profiles may be missed	Add pose-aware detection model
LBPH threshold fixed at 70	Threshold may not suit every camera	Calibrate on held-out data
CSV file storage	No roles, locking or audit trail	Use SQLite or managed database
Single local webcam	No central multi-camera coordination	Add controlled service/API layer
No authentication	Anyone using the PC may access files	Add user accounts and permissions
No formal accuracy study	Performance claims are unsupported	Build labelled evaluation dataset

These are not defects that invalidate the demonstration; they describe the boundary between a learning prototype and a production system. A production version would require stronger security, privacy controls, monitoring, operational support and a formal evaluation across the intended population and camera conditions.

Development priority

The highest-value next steps are a clean unknown-face test, a controlled duplicate-prevention test, and a documented privacy-safe repository review.

PROJECT CHECKPOINT | QUICK REFERENCE

Spoofing

A still photograph may be accepted by a non-liveness prototype.

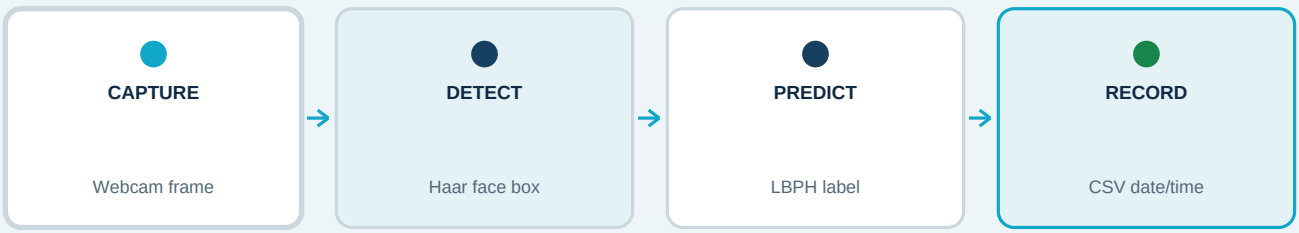
Threshold

A fixed value must be validated before broader use.

Scaling

CSV and a single webcam do not provide multi-user controls.

CORE PROCESSING PATH



TAKEAWAY CSV and a single webcam do not provide multi-user controls.

27. TROUBLESHOOTING GUIDE

Common development issues and resolutions

Symptom	Likely cause	Resolution
No module named cv2	Wrong Python interpreter or missing package	Activate .venv; install requirements there
No module named cv2.face	OpenCV non-contrib build active	Install opencv-contrib-python in the environment
Camera window does not open	Camera busy or permission denied	Close camera apps; check Windows privacy settings
No face rectangles	Poor light, pose or cascade mismatch	Face camera; improve light; check cascade path
Face becomes Unknown	LBPH distance exceeds threshold	Check dataset quality and validate threshold
CSV row not added	Name already in marked_today	Check current date and CSV rows
Wrong folder or file not found	Terminal opened outside project folder	cd to project root before running app.py
Activation script blocked	PowerShell execution policy	Use temporary process-scope bypass for this session

The successful demonstration recording shows an active .venv prompt and the project root in the terminal. If a future launch uses another Python interpreter, VS Code can report that cv2 is missing even though the project environment already has the package. Always check the selected interpreter and terminal prompt first.

Safe PowerShell activation workaround

For the current terminal only: Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass, then run .\.venv\Scripts\Activate.ps1. This does not permanently change the machine's policy.

PROJECT CHECKPOINT | QUICK REFERENCE

Interpreter

Choose the project's .venv rather than global Python.

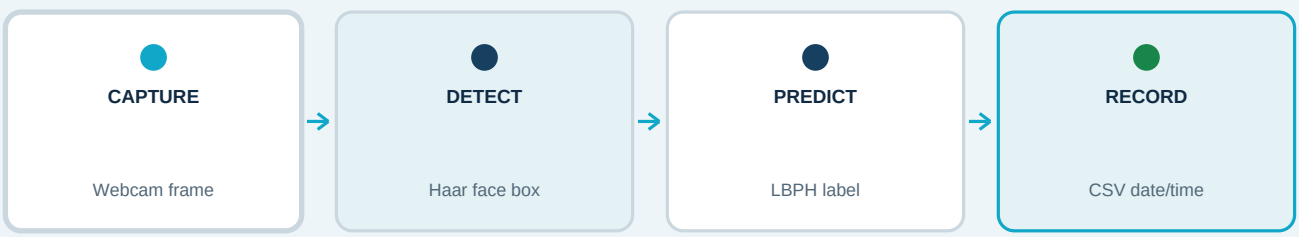
Camera

Check Windows permissions and close other camera apps.

Dataset

Check folder names and image readability if faces are not detected.

CORE PROCESSING PATH



TAKEAWAY Check folder names and image readability if faces are not detected.

28. INSTALLATION AND EXECUTION GUIDE

Reproducible local run steps

ONE-TIME SETUP

PYTHON SOURCE

```
001  cd "C:\Users\P THARUN SAI RAM\Face-Recognition-Attendance-System"
002  py -3.12 -m venv .venv
003  Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
004  .\.venv\Scripts\Activate.ps1
005  python -m pip install --upgrade pip
006  pip install -r requirements.txt
```

RUN THE APPLICATION

PYTHON SOURCE

```
001  cd "C:\Users\P THARUN SAI RAM\Face-Recognition-Attendance-System"
002  .\.venv\Scripts\Activate.ps1
003  python app.py
004  # Press Q in the OpenCV webcam window to exit.
```

The directory and username above match the local environment visible in the user's development screenshots. On another computer, adjust the folder path. Run the command from the folder containing app.py, and confirm that .venv is shown at the beginning of the terminal prompt before starting the application.

Interpreter check

In VS Code, choose the Python interpreter inside the project's .venv. Do not use the global Python interpreter if the project's dependencies are installed only in .venv.

PROJECT CHECKPOINT | QUICK REFERENCE

Folder

Run commands from the project root where app.py exists.

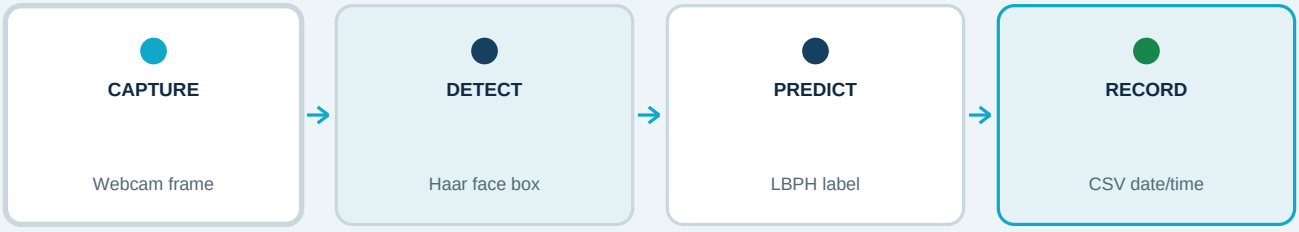
Environment

Activate .venv before installing or running dependencies.

Launch

Use python app.py and press Q to close the webcam window.

CORE PROCESSING PATH



TAKEAWAY Use python app.py and press Q to close the webcam window.

29. SOURCE CONTROL AND GITHUB

Public project repository

The project source is maintained in a public GitHub repository named Face-Recognition-Attendance-System. The repository listing shows app.py, README.md, attendance.csv, requirements.txt and known_faces. The project description identifies it as an automated attendance system using Python, OpenCV and face recognition.

PUBLIC GITHUB REPOSITORY

https://github.com/pendyalatharun14-a11y/Face-Recognition-Attendance-System

RECOMMENDED REPOSITORY CONTENTS

- app.py: source code for detection, recognition and attendance logging.
- requirements.txt: package dependencies required for setup.
- README.md: project purpose, features, installation and execution steps.
- .gitignore: rules for .venv, real face images and personal attendance data.
- Documentation: report PDF and non-sensitive diagrams or screenshots.

KEEP PRIVATE

Do not publish personal face photos or real attendance records merely to make the repository look complete. If the current repository has tracked personal files, review the commit history and remove them safely as appropriate. For public examples, use synthetic names and timestamps or consent-cleared sample imagery.

Public documentation quality

A strong public project page should explain what the software actually does, provide the correct run steps, list dependencies, show genuine screenshots and state current limitations clearly.

PROJECT CHECKPOINT | QUICK REFERENCE

Source

app.py is the main executable file in the repository.

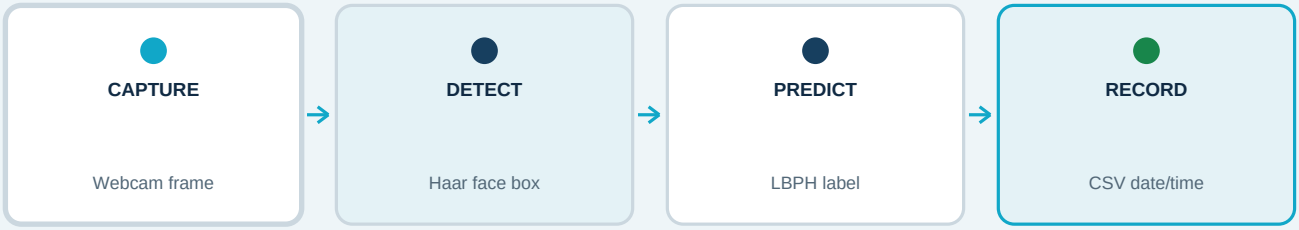
Dependencies

requirements.txt lists the Python packages.

Public share

Review images, CSV rows and ignore rules before every commit.

CORE PROCESSING PATH



TAKEAWAY Review images, CSV rows and ignore rules before every commit.

APPENDIX A: PYTHON SOURCE CODE

Part 1 of 4 - imports, configuration and face dataset preparation

This part defines the project paths, loads the frontal-face cascade, creates the attendance header when needed and begins preparing labelled face samples.

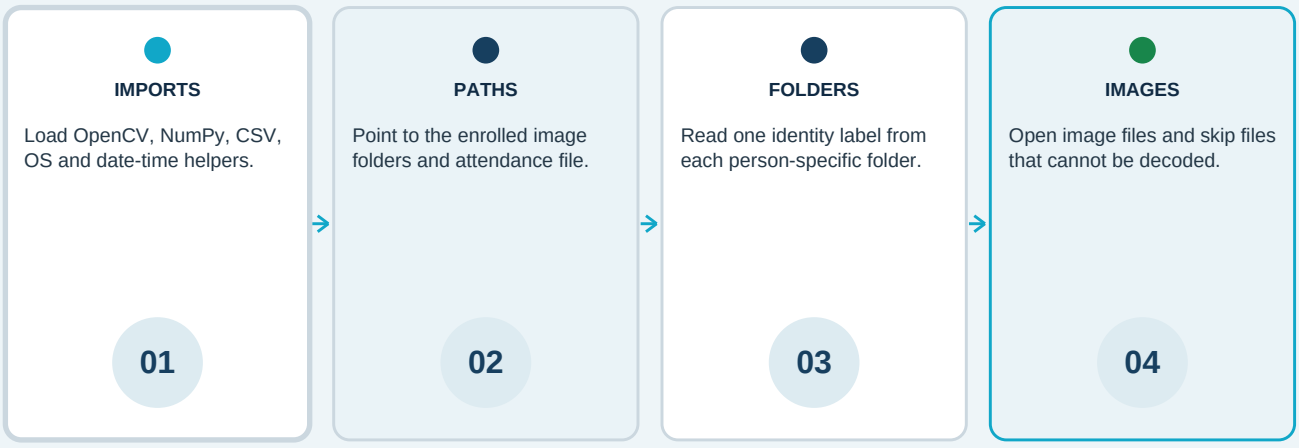
PYTHON SOURCE

```
001  import cv2
002  import os
003  import csv
004  import numpy as np
005  from datetime import datetime
006
007  # Face Recognition Attendance System
008  DATASET_PATH = "known_faces"
009  ATTENDANCE_FILE = "attendance.csv"
010
011  # Load the frontal-face detector provided with OpenCV.
012  face_detector = cv2.CascadeClassifier(
013      cv2.data.haarcascades +
014      "haarcascade_frontalface_default.xml"
015  )
016
017  # Create the attendance file with a header if it is missing.
018  if not os.path.exists(ATTENDANCE_FILE):
019      with open(ATTENDANCE_FILE, "w", newline="") as file:
020          writer = csv.writer(file)
021          writer.writerow(["Name", "Date", "Time"])
022
023  # Create and prepare the LBPH face recogniser.
024  recognizer = cv2.face.LBPHFaceRecognizer_create()
025  faces = []
026  labels = []
027  names = {}
028  label_id = 0
029
030  # Each subfolder name becomes a person label.
031  for person in os.listdir(DATASET_PATH):
032      person_path = os.path.join(DATASET_PATH, person)
033      if not os.path.isdir(person_path):
034          continue
035
036      names[label_id] = person
037
038      # Read all usable images in this person's folder.
```

CODE LISTING - PART 1

Focus: project imports, local paths, Cascade loading and initial dataset traversal.

PROJECT CHECKPOINT | QUICK REFERENCE



PROCESS MAP A compact visual guide to the page's implementation topic.

APPENDIX B: PYTHON SOURCE CODE

Part 2 of 4 - face cropping, labels and LBPH model training

This part completes the training sample preparation, assigns folder labels and trains the LBPH face recogniser.

PYTHON SOURCE

```
        for image_name in os.listdir(person_path):
            image_path = os.path.join(person_path, image_name)
            image = cv2.imread(image_path)
            if image is None:
                continue

            gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
            detected = face_detector.detectMultiScale(gray, 1.3, 5)
            if len(detected) == 0:
                continue

            # Select the largest detected face from the image.
            x, y, w, h = max(
                detected, key=lambda box: box[2] * box[3]
            )
            face = gray[y:y+h, x:x+w]
            face = cv2.resize(face, (200, 200))
            faces.append(face)
            labels.append(label_id)

        label_id += 1

    if len(faces) == 0:
        print("No face images found in known_faces folder.")
        print("Add student photos before running the program.")
        raise SystemExit

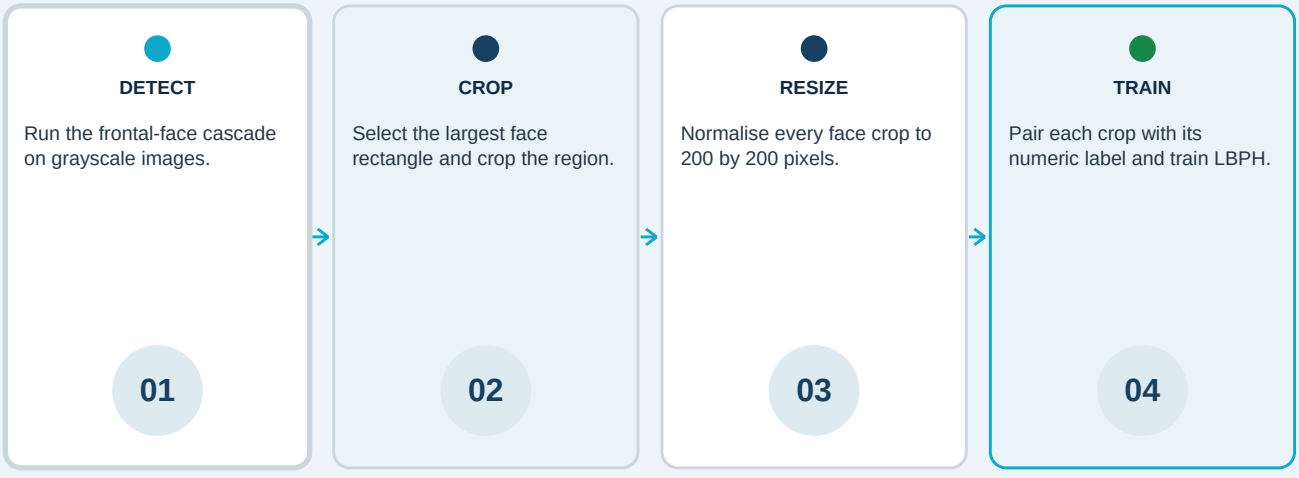
    recognizer.train(
        faces, np.array(labels, dtype=np.int32)
    )
    print("Face Recognition Model Trained Successfully!")

    # Build the set of names already marked on the current date.
    marked_today = set()
    if os.path.exists(ATTENDANCE_FILE):
        with open(ATTENDANCE_FILE, "r", newline="") as file:
            reader = csv.DictReader(file)
            for row in reader:
```

CODE LISTING - PART 2

Focus: normalised face samples, numeric labels, empty-dataset handling and recognizer.train().

PROJECT CHECKPOINT | QUICK REFERENCE



PROCESS MAP A compact visual guide to the page's implementation topic.

APPENDIX C: PYTHON SOURCE CODE

Part 3 of 4 - daily attendance state and webcam processing

This part loads the names already marked today, opens the camera, reads frames, converts them to grayscale and detects face regions.

PYTHON SOURCE

```
            if row.get("Date") == datetime.now().strftime("%Y-%m-%d"):
                marked_today.add(row.get("Name"))

# Start the default webcam.
camera = cv2.VideoCapture(0)
if not camera.isOpened():
    print("Cannot access webcam.")
    raise SystemExit

print("Attendance System Started!")
print("Press Q to exit.")

while True:
    success, frame = camera.read()
    if not success:
        break

    # Convert the current camera frame to grayscale.
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Detect faces in the current frame.
    detected_faces = face_detector.detectMultiScale(
        gray, 1.3, 5
    )

    for (x, y, w, h) in detected_faces:
        face = gray[y:y+h, x:x+w]
        face = cv2.resize(face, (200, 200))

        # LBPH returns a numeric identity label and distance.
        label, confidence = recognizer.predict(face)

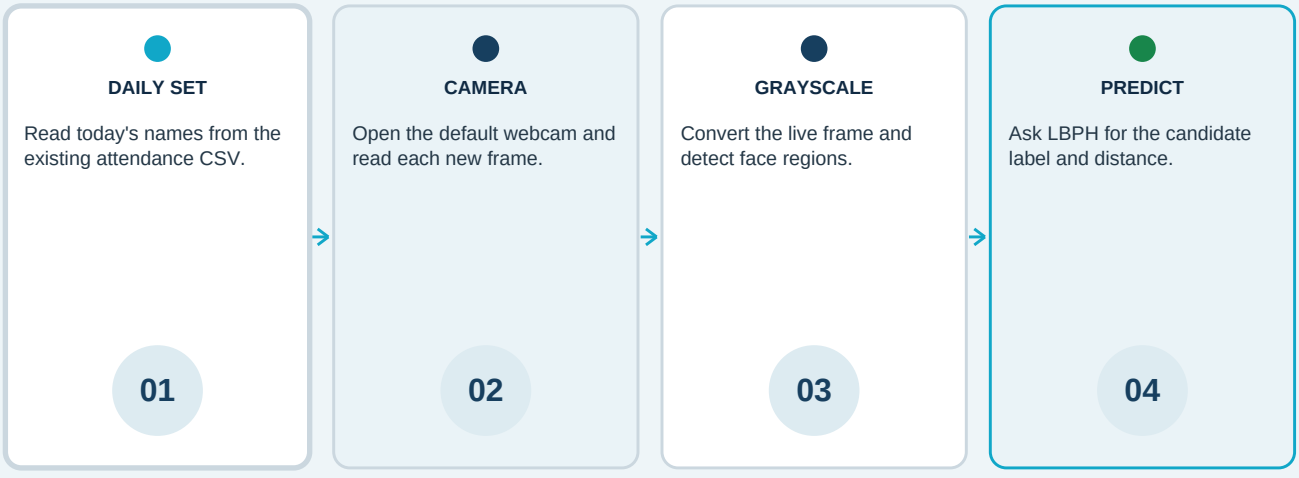
        if confidence < 70 and label in names:
            name = names[label]
            color = (0, 255, 0)

            # Add a row only once for each person per date.
            if name not in marked_today:
```

CODE LISTING - PART 3

Focus: marked_today is populated from CSV before the live webcam loop starts.

PROJECT CHECKPOINT | QUICK REFERENCE



PROCESS MAP A compact visual guide to the page's implementation topic.

APPENDIX D: PYTHON SOURCE CODE

Part 4 of 4 - prediction, attendance writing and window cleanup

This part performs the LBPH prediction, displays known or Unknown labels, appends first-daily attendance rows and releases the camera.

PYTHON SOURCE

```
                now = datetime.now()
                with open(
                    ATTENDANCE_FILE, "a", newline=""
                ) as file:
                    writer = csv.writer(file)
                    writer.writerow([
                        name,
                        now.strftime("%Y-%m-%d"),
                        now.strftime("%H:%M:%S")
                    ])

                marked_today.add(name)
                print(name, "Attendance Marked!")
        else:
            name = "Unknown"
            color = (0, 0, 255)

        # Draw a rectangle and the predicted name.
        cv2.rectangle(
            frame, (x, y), (x+w, y+h), color, 2
        )
        cv2.putText(
            frame, name, (x, y-10),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.8, color, 2
        )

    # Display the annotated camera frame.
    cv2.imshow(
        "Face Recognition Attendance System", frame
    )

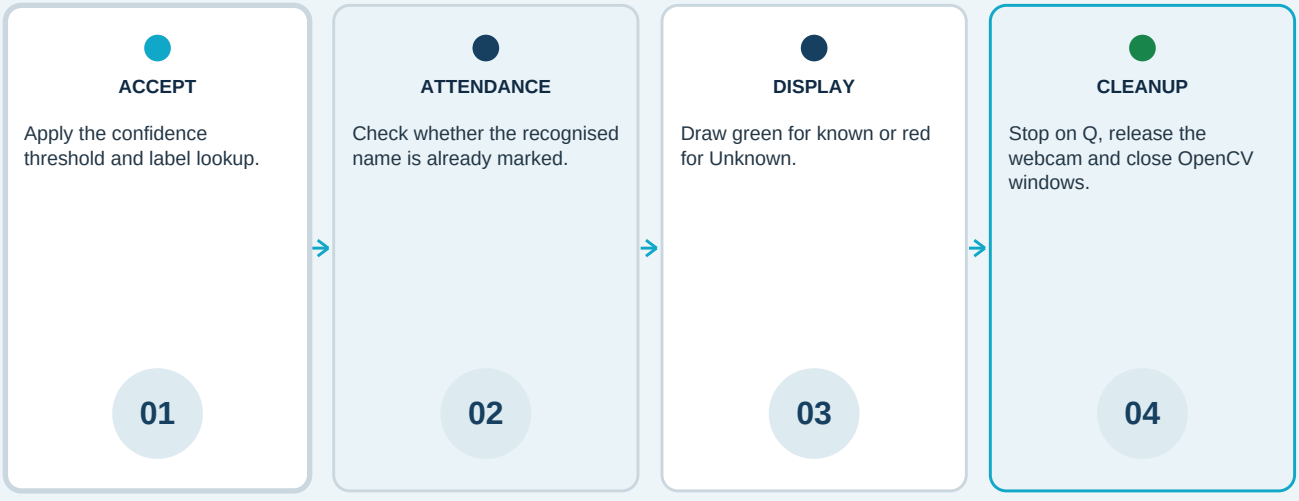
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

camera.release()
cv2.destroyAllWindows()
```

CODE LISTING - PART 4

Focus: confidence threshold, green/red display state, CSV append, Q-key exit and cleanup.

PROJECT CHECKPOINT | QUICK REFERENCE



PROCESS MAP A compact visual guide to the page's implementation topic.

APPENDIX C: CODE WALKTHROUGH

Important functions, variables and control flow

Code item	Role in the program
DATASET_PATH	Folder containing one subfolder per enrolled person
ATTENDANCE_FILE	CSV path used for persistent attendance rows
face_detector	Haar Cascade object used for face-region detection
recognizer	LBPH model trained from grayscale face crops
faces / labels	Training samples and associated numeric labels
names	Map from model label number to person's folder name
marked_today	Set used to block repeat attendance for the current date
VideoCapture(0)	Opens the default webcam
detectMultiScale	Finds candidate face rectangles in grayscale frames
recognizer.predict	Returns predicted label and LBPH distance
cv2.imshow / waitKey	Displays the annotated frame and checks for Q

MAIN BRANCH LOGIC

Accepted known face: confidence is below 70 and the predicted label is present in names. The program retrieves the folder name, draws a green box, and checks the daily set before appending attendance.

Unknown face: if the threshold or label check fails, the name is set to Unknown and the box colour is red. The Unknown branch does not write a CSV record.

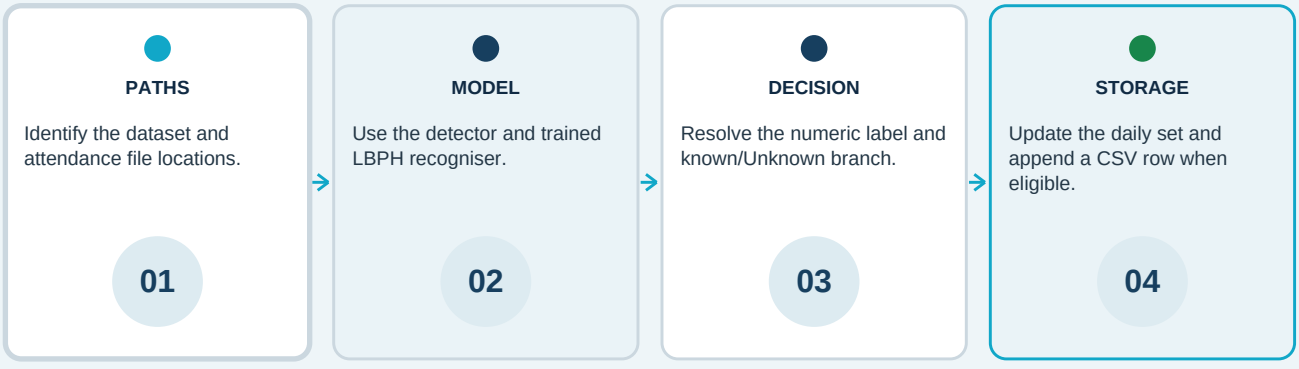
IMPLEMENTATION NOTE

The code is intentionally a single-file prototype. For maintainability, the next version could split enrollment, model training, camera capture, attendance storage and user-interface code into separate modules. This would make each component easier to test without changing the core project concept.

Authoritative source

Repository: https://github.com/pendyalatharun14-a11y/Face-Recognition-Attendance-System | File: app.py | Branch checked for this report: main | Accessed 11 November 2026.

PROJECT CHECKPOINT | QUICK REFERENCE



PROCESS MAP A compact visual guide to the page's implementation topic.

30. FUTURE SCOPE AND CONCLUSION

Next improvements and final project summary

FUTURE ENHANCEMENTS

- **Unknown-face demonstration:** record a separate webcam test using an image kept outside the training dataset.
- **Threshold evaluation:** build a held-out test set and measure false accepts and false rejects before selecting a threshold.
- **Duplicate test:** run two scans on the same date and confirm the CSV gains only one new row.
- **Safer storage:** move from a flat CSV to SQLite with controlled access and backups.
- **Privacy protections:** add consent, retention and deletion procedures for enrolled photos.
- **Liveness protection:** evaluate a suitable anti-spoofing approach before any consequential use.
- **Interface improvements:** add clear status messages and a reporting view after the core behaviour is validated.

CONCLUSION

The project demonstrates the connection of image acquisition, face detection, face recognition and local attendance storage in one Python application. The new 11 November 2026 screen recording shows the model-training success message, the live webcam window with the recognised label Tharun, and the attendance.csv file with date-and-time rows. These are meaningful functional milestones for a personal learning project.

The report avoids unsupported accuracy claims. The implementation remains a small local prototype with a fixed LBPH threshold, simple CSV storage, no liveness detection and no production access-control layer. Further development should focus first on repeatable known/unknown testing, reliable duplicate verification, and careful handling of face photographs and attendance data.

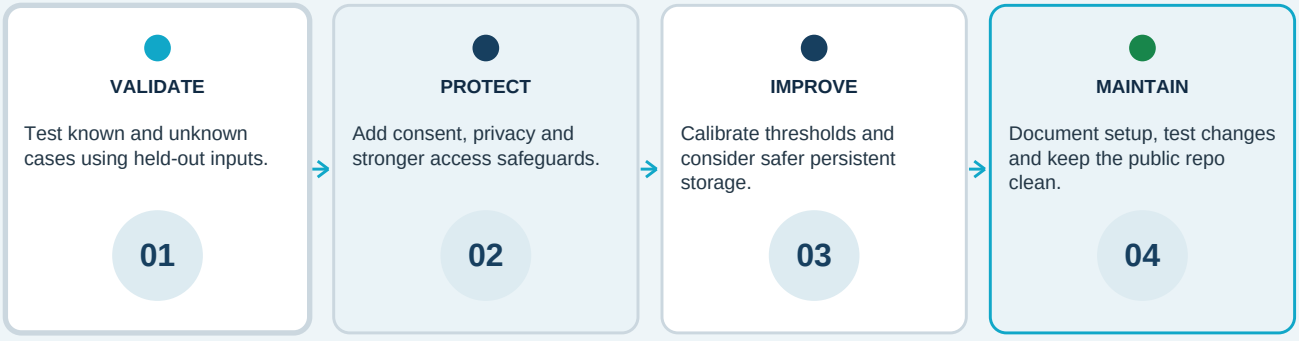
REFERENCES AND PROJECT LINKS

- Project source repository: https://github.com/pendyalatharun14-a11y/Face-Recognition-Attendance-System
- OpenCV documentation: https://docs.opencv.org/
- Python documentation: https://docs.python.org/3/
- Git documentation: https://git-scm.com/doc
- Requirements file in repository: requirements.txt

Final note

This document is personal project documentation. It is not a college approval certificate, a formal biometric security assessment or a measured accuracy report.

PROJECT CHECKPOINT | QUICK REFERENCE



PROCESS MAP A compact visual guide to the page's implementation topic.